

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Bhubaneswar, Odisha

AI SHOPPING HACKATHON 2026

AI Shopping Agent

Conversational Commerce Across Multiple Categories

React · Node.js · Firebase · Shopify API · Groq AI · Voice Assistant

TECHNICAL DOCUMENTATION

TEAM

Aarushi Sharma · MD. Khustar Noorani

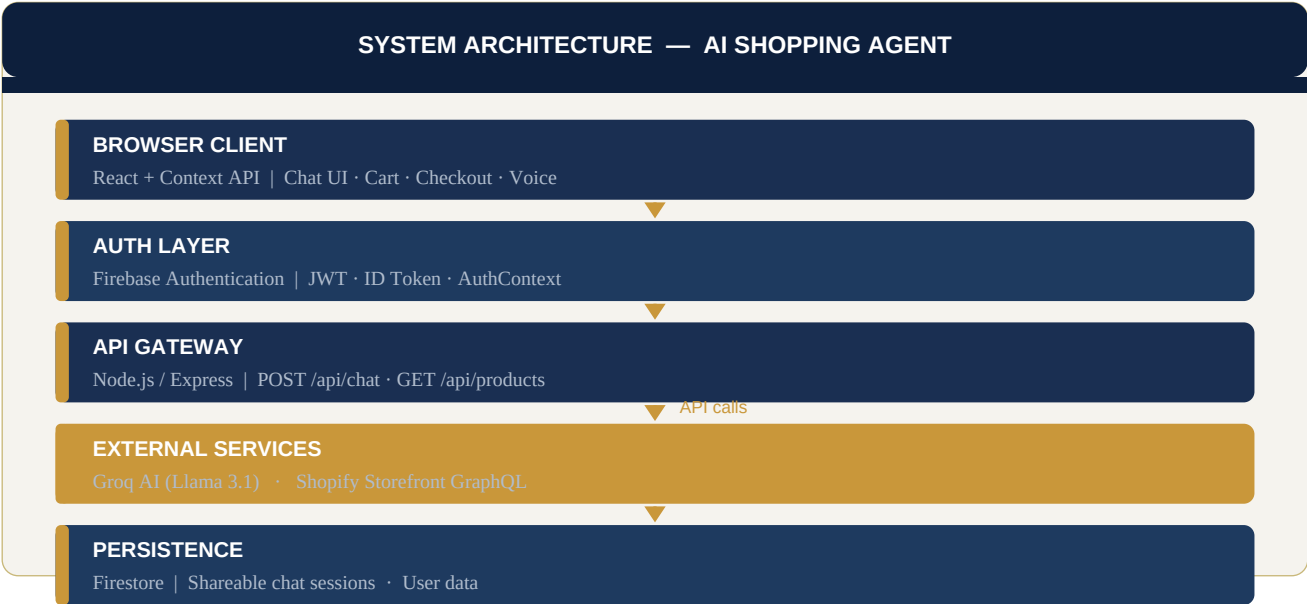
github.com/aarushhii30/AI-shopping-agent

Submitted for: Shopify AI Shopping Hackathon 2026

Category: Conversational Commerce / AI Shopping Assistant

1 SYSTEM ARCHITECTURE

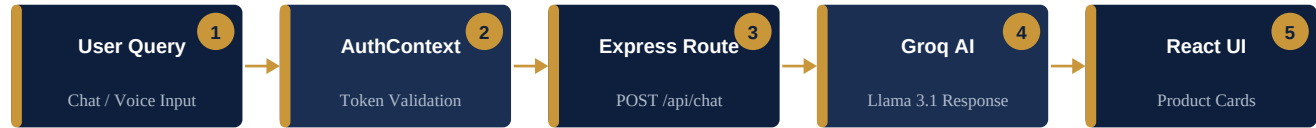
The system is structured in five bounded layers, each with a single responsibility. The AI recommendation engine and Shopify catalog integration live entirely server-side; the React client never calls external APIs directly. Credentials, tokens, and third-party keys are isolated to the server layer throughout.



■ Component Responsibilities

Layer	Technology	Responsibility
Browser Client	React + Context API	Chat UI, Cart State, Checkout, Voice Input (Web Speech API)
Auth Layer	Firebase Auth	Email/password login, JWT issuance, token refresh, AuthContext provider
API Gateway	Node.js / Express	/api/chat forwards to Groq AI; /api/products queries Shopify GraphQL
AI + Commerce	Groq (Llama 3.1) + Shopify	Intent parsing, product ranking; Storefront GraphQL catalog fetch
Persistence	Firestore	Shareable chat session documents keyed by session ID

■ Data Flow



Query lifecycle: chat input → auth → Express → Groq AI → rendered product cards

A user query travels from the chat input through AuthContext token validation to the Express POST /api/chat route, which appends a system prompt and forwards to Groq. The AI returns an intent-parsed response with a structured product list rendered as cards in the chat view. Cart operations are handled entirely client-side via CartContext — no server round-trip until checkout. Shopify credentials never leave the server layer.

■ Voice Input Architecture

Voice uses the browser-native Web Speech API with zero additional infrastructure. The SpeechRecognition object transcribes audio and fires a result event that populates the chat input identically to a typed message, entering the same AI pipeline. On unsupported browsers (Firefox, Safari), a clear tooltip is surfaced rather than silent degradation.

2 KEY IMPLEMENTATION DECISIONS

Every architectural choice was driven by one constraint: deliver a working, demonstrable product within the hackathon window. The decisions below are the ones with the most architectural consequence.

■ React Context API over Redux

Three bounded contexts (auth, cart, checkout) do not justify Redux boilerplate. Context API with custom hooks delivered identical capability in a fraction of setup time.

Tradeoff: Less DevTools visibility; acceptable for this scope.

■ Firebase Auth over custom JWT backend

Building secure auth (bcrypt, refresh tokens, session invalidation) costs a full day. Firebase handles token management, expiry, and session persistence out of the box.

Tradeoff: Vendor dependency on Google auth; manageable at this scale.

■ Shopify Storefront GraphQL over REST Admin API

The Admin API requires admin credentials and is not designed for customer-facing apps. Storefront API is scoped to read-only catalog access. GraphQL eliminates over-fetching.

Tradeoff: Stricter rate limits; addressed with a skeleton loading state.

■ Groq (Llama 3.1) over OpenAI GPT-4o

Groq inference speed makes 600ms responses feel conversational. GPT-4o at 3s breaks flow. Groq's free tier covered hackathon load with no API cost pressure.

Tradeoff: Marginally weaker structured JSON output; mitigated by explicit prompt formatting.

■ Web Speech API over Whisper

Whisper requires a server round-trip (record, upload, transcribe, return). Web Speech API is in-browser, returns transcription in under 200ms, zero cost.

Tradeoff: Chrome/Edge only; Firefox and Safari users see a clear fallback tooltip.

■ Firestore for shareable sessions

Chat history is stored in a Firestore document keyed by session ID. Sharing = sharing a URL. Firestore's schemaless model fits the variable message structure cleanly.

Tradeoff: No access control on shared sessions; acceptable for a discovery demo.

■ Multi-category over single-category

We validated whether one AI interface could span gadgets, fashion, and home decor without per-category prompt engineering. The system prompt instructs the model to detect category from the query.

Tradeoff: No category-specific features (outfit matching, spec comparison) in v1.

3 AI VS DETERMINISTIC LOGIC

The boundary between AI and deterministic code was drawn deliberately: AI handles anything requiring interpretation of ambiguous natural language or subjective ranking. Every calculation, state change, and data mutation is deterministic — predictable, testable, auditable. AI outputs are probabilistic; letting the model control cart arithmetic or checkout routing would introduce unpredictable failure modes in the most trust-sensitive part of the flow.

AI Layer handles	Deterministic Code handles
— Intent understanding from vague queries — "something for the gym" -> fitness products — Recommendation ranking by expressed need — Conversational response generation — Category detection from free-text input — AI match-reason label on each product card	— Cart: add / remove / quantity update — GST calculation (18% on subtotal) — Promo code validation and discount logic — Checkout form validation and step routing — Firebase auth: login, logout, token refresh — Shopify product fetch and normalisation — React Router navigation + protected routes — Firestore read/write for shareable sessions

4 FAILURE HANDLING & SYSTEM RESILIENCE

The guiding principle: never leave the user in a broken or ambiguous state. Each external dependency has an explicit failure path. The question "what happens when the LLM returns garbage?" has a concrete answer for every scenario — the system retries once, then shows a meaningful fallback, never a blank screen.

■ Shopify API failure or timeout

- Express wraps all Shopify calls in try/catch with a 5-second timeout.
- On failure, server returns HTTP 503 with a structured error payload.
- Client shows inline chat message: "I had trouble fetching products — please try again."
- Chat session stays intact; user can retry without losing context.
- Production path: exponential backoff (3 retries) before surfacing the error.

■ AI returns malformed or empty output

- Response is validated for expected JSON structure before any rendering.
- If empty or unparseable: retry once automatically, then show fallback message.
- Fallback: "I couldn't understand that — could you rephrase?"
- Product card rendering only runs if a valid product array is present.
- Raw AI output is never exposed to the user under any code path.

■ **Firestore authentication failure**

- All protected routes check for a valid Firestore ID token on mount.
- Expired or missing token redirects to /login — no silent access.
- Wrong password / unregistered email surface as inline field errors, not crashes.
- Network failure during login shows a retry prompt, not a blank screen.

■ **Unsupported browser for voice input**

- Voice button renders conditionally on window.SpeechRecognition availability.
- On unsupported browsers: mic icon replaced with tooltip "Voice not supported. Try Chrome."
- No silent degradation — feature limitation is clearly communicated.

■ **Malformed or empty user query**

- Queries under 2 characters are blocked at the input level before any API call.
- System prompt instructs Groq to ask a clarifying question rather than return zero results.
- Empty Shopify result set renders an inline message, not an empty product grid.

■ **Slow API responses (3-5 seconds)**

- Typing animation plays in chat while AI response is pending.
- Skeleton card grid shown while products are loading.
- If no response within 10 seconds, a timeout message is surfaced.
- Framer Motion transitions make loading states feel intentional, not broken.

■ **Checkout state integrity**

- CheckoutContext validates each step before allowing progression.
- Shipping details required before payment step is accessible.
- Navigating directly to /checkout without cart items redirects to cart.
- No payment is processed in this build, so no financial failure modes exist.

5 **KNOWN LIMITATIONS & WHAT WE WOULD IMPROVE**

These are not oversights — they are conscious scope decisions made given the hackathon timeline. Understanding what a system does not do is as important as what it does. Each limitation below is paired with a concrete improvement path.

Limitation	Current State	Improvement Path
No real payment processing	Checkout UI supports Card, UPI, PayPal, Apple/Google Pay selection but no gateway is connected.	Integrate Razorpay (India) or Stripe; form fields already map to their SDK inputs.
No vector / semantic search	Product search uses keyword-based Shopify Storefront queries only.	Add Pinecone or Weaviate; embed product descriptions; enable semantic matching ("something minimalist").
No persistent memory	Each session is stateless at the AI layer. No cross-session preference retention.	Store a user preference vector in Firestore; inject as context on session start.
Voice: Chrome / Edge only	Web Speech API not standardised; Firefox and Safari users cannot use voice.	Server-side Whisper transcription endpoint solves browser support at the cost of added latency.
No autonomous purchasing	AI recommends and guides but cannot complete a purchase on behalf of the user.	Requires persistent auth, stored payment methods, and explicit user trust delegation.
English-only UI	Interface, prompt chips, and system prompts are English only.	Language detection step + localised system prompts; Llama 3.1 already supports Hindi, Tamil, Bengali.
No category-specific AI tuning	Single system prompt handles all categories (gadgets, fashion, home decor).	Per-category prompt strategies: outfit matching for fashion, spec comparison for gadgets.

6 PERFORMANCE, SECURITY & ROADMAP

■ Performance Bottlenecks

- Shopify latency (3-5s): Redis cache with 5-min TTL on category-level queries eliminates repeat round-trips.
- AI token overhead: System prompt sent every query. Production fix: initialise once via Groq context window.
- Bundle size: Framer Motion adds ~40KB. Code splitting per route + lazy checkout reduces initial load.

■ Security Posture

- All API keys (Shopify, Groq, Firebase Admin) in git-ignored .env files. Zero credentials committed.
- React Router PrivateRoute blocks all authenticated routes — no client-side bypass path.

- Server-side Firebase Admin SDK verifies ID tokens before processing any request.
- Shopify Storefront token never reaches the client. All catalog queries proxied through Express.
- User inputs trimmed, length-capped, and parameterised before any GraphQL interpolation.

■ Roadmap Priority Order

Phase	Focus	Key Deliverables
Phase 1	Intelligence depth	Vector search (Pinecone), per-category prompts, persistent user profiles
Phase 2	Commerce completeness	Razorpay/Stripe gateway, conversational checkout, order history
Phase 3	Platform features	Image-based shopping, AI product comparison, virtual try-on (AR), multilingual
Phase 4	Merchant layer	Analytics dashboard, autonomous reorder (trusted users), multi-store support

7 CONCLUSION

What we built over this hackathon is not a demo dressed up as a product — it is a functional conversational commerce layer with real catalog data, real authentication, and a complete purchase flow. The architecture is clean enough to extend: swap the AI provider, add a vector store, integrate a payment SDK — each is an additive change, not a rewrite.

The core thesis holds: users who do not know what they want are better served by a conversation than a filter tree. The Groq-powered engine handles vague, cross-category intent that no traditional search bar can resolve. Voice input, shareable sessions, and failure-resilient code are the thesis in practice.

The current build proves that conversational commerce works and checkout integrates cleanly. The next phase is going deeper on intelligence, not wider on features.